AI Security Harness Lab

Installation, smoke-testing and capability review of 19 open-source AI security repositories

Mohammad Ibrahim Saleem | 2026-10-14

Metric	Result
Repositories cloned	19 / 19 (SHA, license, release recorded)
Installed in isolated environments	16
Passed safe smoke tests	16 (38 sandboxed L1/L2 runs + 2 local-model L3 runs)
Blocked	Shannon (needs approval to run npx), OWASP ZAP (no Java, Docker off)
Config-validated only	PentAGI (root + host docker.sock by default)
Network during L1/L2 tests	None (sandbox-exec denied all IP traffic)
Paid AI tokens consumed	0

1. Executive summary

- All 19 repositories were cloned; every license was reviewed (VVAH and AutoGen corrected by hand).
- 16 tools installed into separate, lab-local environments. Nothing system-wide, no sudo, no remote install scripts.
- Every test ran under macOS sandbox-exec with network and socket binding denied and a fake $HOME, so real credentials and CLI logins were unreachable.
- Semgrep and Opengrep each found 12/12 seeded issues in a synthetic fixture, identically. These are the only verified security findings.
- A local llama3.1:8b model resisted 5/5 Promptfoo secret-leak attempts but was hijacked by 55% of Garak prompt-injection probes (20-prompt sample).
- No active penetration tests, exploit verification or cloud-model calls were run.

2. System

Item	Value
OS / arch	macOS 27.0.1, arm64
CPU / GPU	Apple M5 Pro, 15 CPU cores, 16 GPU cores (Metal 4, no CUDA)
RAM / disk	24 GB unified / 732 GB free at start; lab uses ~24 GB incl. model
Python	3.12 (pyenv) + lab-local 3.10, 3.11, 3.13 via uv
Node / npm	24.21.0 / 11.19.0
Docker	CLI 29.8.0, Compose v5.5.1; daemon not running
Missing	Java 17+, Poetry, pipx, Go, cosign
Local LLM	Ollama 0.34.4 on 127.0.0.1:11434 with llama3.1:8b

3. Isolation and safety controls

- Interpreters, venvs, uv tools, npm packages and caches all under `environments/` (`configs/lab.env`).
- Test runner `configs/smoke.sh`: sandbox-exec profile denies outbound IP, bind and inbound; Level 3 profile allows only localhost:11434.
- Tests requiring localhost services, Docker, or live model backends were excluded until approved.
- Only touch outside the lab: three uv Python symlinks briefly created in ~/.local/bin and removed immediately; ~/.cache/uv gained cache entries.

4. All repositories at a glance

ID	Project	License	Version	Status	Last commit
H01	Visa Vulnerability Agentic Harness (VVAH)	Apache-2.0*	1.5.0	PASS	2026-10-09
H02	VulnHuntr	AGPL-3.0	0.1.0	PASS	2025-02-06
H03	Strix	Apache-2.0	1.7.0	PASS (L1)	2026-10-14
H04	PentAGI	MIT	v2.2.0 (repo)	CONFIG ONLY	2026-10-10
H05	Shannon	AGPL-3.0	v3.4.0 (repo)	BLOCKED	2026-10-07
H06	PentestGPT	MIT	1.0.0	PASS	2026-07-14
H07	SWE-agent	MIT	1.1.0	PASS (L1)	2026-07-16
H08	Inspect AI (UK AISI)	MIT	0.3.280	PASS	2026-10-14
H09	Promptfoo	MIT	0.124.1	PASS	2026-10-13
H10	NVIDIA Garak	Apache-2.0	0.17.1.pre1	PASS	2026-10-10
H11	Microsoft PyRIT	MIT	1.2.0.dev0	PASS	2026-10-14
H12	OpenHands	MIT	agent-canvas 1.27.1	PASS (L1)	2026-10-13
H13	Aider	Apache-2.0	0.86.2	PASS	2026-05-22
H14	Semgrep	LGPL-2.1	1.180.0	PASS	2026-10-07
H15	Opengrep	LGPL-2.1	1.30.2	PASS	2026-10-10
H16	OWASP ZAP	Apache-2.0	v2.17.0 (repo)	BLOCKED	2026-10-08
H17	LangGraph	MIT	1.2.15	PASS	2026-10-13
H18	Microsoft AutoGen	CC-BY-4.0	0.7.5	PASS	2026-04-06
H19	CrewAI	MIT	1.15.27	PASS	2026-10-14

* GitHub reports NOASSERTION; LICENSE file is Apache-2.0. AutoGen code is MIT (docs CC-BY-4.0).

5. Repository and harness details

H01 - Visa Vulnerability Agentic Harness (VVAH)

Primary AI security harness Status: **PASS**

Purpose	Agentic source-code security analysis: attack-surface/threat modelling, vulnerability discovery, adversarial finding verification, structured reports and SARIF. Optional remediation (S10), remediation validation (S11) and exploit verification, all off by default.
Repository	https://github.com/visa/visa-vulnerability-agentic-harness branch `main` commit `4128e870377850db16cb082115105cc8ee6f1a72`
License	Apache-2.0 (LICENSE text reviewed by hand; GitHub API reports NOASSERTION).
Release / activity	Latest release: v1.5.0 (2026-10-09). Last commit 2026-10-09. Stars: 2928. Very active. v1.5.0 released 2026-10-09.
Installation	Python 3.11 venv `environments/vvah`; `pip install .` from source.
Installed version	1.5.0
Tests run	- L1 `vvaharness --version` / `--help`: pass. - L2 `vvaharness setup`: readiness check works. Exit 1 = 'Not ready: 3 blocking', all of which are missing API keys (expected). - L2 `vvaharness estimate` on synthetic fixture: 7 files, ~507 source tokens, S1-S9 budget 252k-1M input tokens. No model calls.
AI models	Anthropic; any OpenAI-compatible endpoint (incl. open-weight models); can drive Claude Code / Codex / Gemini CLIs. No native Ollama provider.
Capabilities	Vulnerability discovery, LLM + tree-sitter static analysis, per-role multi-agent pipeline, SARIF/JSON/MD output, optional exploit verification.
Risks	Sends source code to the model provider. Auto-detects a logged-in claude CLI. Remediation edits source, so use on copies only.
Next step	Choose a model and spend cap, then `vvaharness scan --repo fixtures/sample-vulnerable-app --stop-after s9`.

H02 - VulnHuntr

Primary AI security harness Status: **PASS**

Purpose	LLM-assisted discovery of remotely exploitable vulnerabilities in Python code by tracing call chains from user input to sinks.
Repository	https://github.com/protectai/vulnhuntr branch `main` commit `ead88c5adba4279dae5c56d65124c530a9a1c5ae`
License	AGPL-3.0 (strong copyleft).
Release / activity	Latest release: none. Last commit 2025-02-06. Stars: 2801. Stale: last commit 2025-02-06, no GitHub releases.
Installation	Python 3.10 venv `environments/vulnhuntr`; hash-pinned `requirements.txt` + `--no-deps` source install (Poetry absent).
Installed version	0.1.0
Tests run	- L1 `vulnhuntr --help`: pass after fix. A plain `pip install .` crashes because `requests` is not declared as a dependency (upstream packaging gap).
AI models	Claude, GPT, Ollama (`-l {claude,gpt,ollama}`), optional custom endpoint.
Capabilities	Python only. Classes: LFI, arbitrary file overwrite, RCE, XSS, SQLi, SSRF, IDOR. Text/log output (no SARIF).
Risks	Sends source to model provider. AGPL licensing.
Next step	Optional second opinion for Python repos only; low priority because upstream is stale.

H03 - Strix

Primary AI security harness Status: **PASS (L1)**

Purpose	Agentic application security testing: AI agents run real tools inside a Docker sandbox against a target app.
Repository	https://github.com/usestrix/strix branch `main` commit `956a26715b02ec2d60c6a6ffcf9c6146633a4318`
License	Apache-2.0.
Release / activity	Latest release: v1.7.0 (2026-10-05). Last commit 2026-10-14. Stars: 68421. Very active (commits on clone day). v1.7.0 2026-10-05. Also offers a hosted service (not used).
Installation	`uv tool install strix-agent` (lab-scoped tool dir). Remote shell installer not used.
Installed version	1.7.0
Tests run	- L1 `strix --help` and `--version`: pass.
AI models	Many providers via LiteLLM, incl. Ollama.
Capabilities	Dynamic testing, multi-agent, run reports; MCP and SARIF references in source; CI mode documented.
Risks	Active testing tool. Needs Docker and pulls a sandbox image on first run. Only against isolated, authorised targets.
Next step	Start Docker Desktop; Level-4 approval with an isolated target before any `--target` run.

H04 - PentAGI

Primary AI security harness Status: **CONFIG ONLY**

Purpose	Multi-agent autonomous penetration-testing platform with web UI, running tools in containers.
Repository	https://github.com/vxcontrol/pentagi branch `main` commit `843dfeb4bfc809c09d1bc5022e567c602d8fc9dd`
License	MIT for code, plus a separate EULA.md (review before commercial use).
Release / activity	Latest release: v2.2.0 (2026-10-05). Last commit 2026-10-10. Stars: 28309. Active. v2.2.0 2026-10-05.
Installation	Docker Compose only; not deployed.
Installed version	v2.2.0 (repo)
Tests run	- L2 `docker compose --env-file .env.example config --quiet`: validates. - Runtime NOT TESTED: Docker daemon off and deployment requires approval.
AI models	Many providers; Ollama heavily referenced in source.
Capabilities	Autonomous pentest, multi-agent, GraphQL/REST API, Langfuse/observability stacks.
Risks	Default compose mounts `/var/run/docker.sock` into a container running as root (effectively root on the Docker VM). Requires >= 20 GB disk, heavy RAM.
Next step	Approve Docker start and a socket-proxy / separate-VM design before deployment.

H05 - Shannon

Primary AI security harness Status: **BLOCKED**

Purpose	Autonomous white-box application penetration testing that actively executes exploits.
Repository	https://github.com/KeygraphHQ/shannon branch `main` commit `2468b67d9e40d70d0b189462c5041c24e1dc886c`
License	AGPL-3.0.
Release / activity	Latest release: v3.4.0 (2026-10-06). Last commit 2026-10-07. Stars: 48858. Active. v3.4.0 2026-10-06.
Installation	Not installed: documented entry point `npx @keygraph/shannon@latest` downloads and executes code.
Installed version	v3.4.0 (repo)
Tests run	- L2 `docker compose config --quiet`: validates (Temporal service, named volume only, no socket mount).
AI models	Anthropic primarily; a few Ollama / LiteLLM references.
Capabilities	Dynamic exploitation, source-aware testing, Temporal workflows, Markdown reports; SARIF references in source.
Risks	Executes real exploits. Never point at production, public targets or the host. AGPL.
Next step	Approve a pinned run, e.g. `npx @keygraph/shannon@3.4.0 --help`; needs Docker + LLM credentials.

H06 - PentestGPT

Primary AI security harness Status: **PASS**

Purpose	AI-assisted penetration-testing and CTF workflows. Agent mode drives Claude Code or Codex CLIs; legacy interactive mode supports many LLMs.
Repository	https://github.com/GreyDGL/PentestGPT branch `main` commit `e8b1bb77d1ac00329675cec3b060aba971ec1ac8`
License	MIT.
Release / activity	Latest release: v1.0.0 (2025-12-24). Last commit 2026-07-14. Stars: 15933. Moderately active: last commit 2026-07-14, v1.0.0 2025-12-24.
Installation	`make install` (reviewed: runs `uv sync`) creating `repos/PentestGPT/.venv` and `pentestgpt_agent/.venv`, Python 3.12.
Installed version	1.0.0
Tests run	- L1 `pentestgpt-legacy --help`: pass. - L2 root unit tests: 133 passed. L2 agent unit tests: 115 passed. - Excluded: `live` marker (spends tokens), `docker` marker, and tests that start a vulnerable SSTI server on localhost and attack it.
AI models	Claude Code / Codex CLI backends; legacy: OpenAI, Gemini, DeepSeek, xAI, Qwen, Moonshot, local Ollama.
Capabilities	Dynamic pentest/CTF, supervisor/executor loop, MCP tool server (unit-tested).
Risks	Uses ambient Claude/Codex logins. Git dependency pulled from a second org (PentestGPT-Project/UnifedAgentWrapper, pinned by SHA).
Next step	Level-4 only, against an isolated CTF-style target.

H07 - SWE-agent

Primary AI security harness Status: **PASS (L1)**

Purpose	Software-engineering agent for issue resolution; useful for reviewing vulnerability reports and drafting candidate fixes.
Repository	https://github.com/SWE-agent/SWE-agent branch `main` commit `3ea751c087f32b16e039a2233dd6eefecef325d5`
License	MIT.
Release / activity	Latest release: v1.1.0 (2025-05-22). Last commit 2026-07-16. Stars: 20545. Last commit 2026-07-16; last release v1.1.0 (2025-05). README says most effort moved to mini-swe-agent.
Installation	Python 3.11 venv `environments/swe-agent`; `pip install -e .`
Installed version	1.1.0
Tests run	- L1 `sweagent --help`: pass.
AI models	Any provider via LiteLLM, incl. local.
Capabilities	Patch generation, trajectories, batch mode.
Risks	Edits code. Runs need Docker (SWE-ReX sandbox).
Next step	With Docker on, patch-only runs on a copy of the fixture.

H08 - Inspect AI (UK AISI)

Primary AI security harness Status: **PASS**

Purpose	Framework for evaluating models and agents: datasets, solvers, scorers, sandboxed tools, approval policies, log viewer.
Repository	https://github.com/UKGovernmentBEIS/inspect_ai branch `main` commit `dfcb169e1cb170a8fbe2c37a0e3befc45c2c2b85 (release tag 0.3.280)`
License	MIT.
Release / activity	Latest release: 0.3.280 (2026-10-14). Last commit 2026-10-14. Stars: 2997. Very active (release 0.3.280 on clone day).
Installation	Python 3.12 venv `environments/inspect_ai`; `pip install -e .[dev]` at tag 0.3.280.
Installed version	0.3.280
Tests run	- Install fix: shallow clone had no tags, so setuptools-scm reported 0.1.dev1 and the [dev] resolve failed. Fixed by checking out the release tag. - L1 `inspect --version` / `--help`: pass. - L2 mock-LLM eval: first runs exited 0 but log status was **error** (tiktoken download blocked). After pre-seeding the cache: status success, accuracy 1.0.
AI models	Nearly all providers, incl. Ollama and vLLM.
Capabilities	LLM/agent evaluation, agent solvers, MCP support, .eval logs; ideal scoring layer.
Risks	Paid evals cost tokens; keep to local models unless approved.
Next step	Use as the common scorer to compare tools across runs.

H09 - Promptfoo

Primary AI security harness Status: **PASS**

Purpose	LLM application evaluation and red-team testing (prompt injection, jailbreaks, PII leakage, etc.).
Repository	https://github.com/promptfoo/promptfoo branch `main` commit `a32c0882440adfc70aaca2c16280247261c4c03c`
License	MIT.
Release / activity	Latest release: code-scan-action-0.2.2 (2026-10-08). Last commit 2026-10-13. Stars: 25982. Very active.
Installation	npm `--prefix environments/promptfoo`, pinned 0.124.1; telemetry and update checks disabled.
Installed version	0.124.1
Tests run	- L1 `--version` / `--help`: pass. - L2 echo-provider eval: 1/1 assertions, 0 tokens. - L3 local llama3.1:8b: 5/5 secret-leak tests passed (no leak), 351 local tokens.
AI models	Nearly all providers, incl. Ollama.
Capabilities	Red-team plugins, regression tests, JSON/HTML/CSV output, CI/GitHub Action.
Risks	Model-graded asserts default to a cloud grader model; use deterministic asserts or a local grader.
Next step	Point at an LLM application you own; add red-team plugin suite with a local grader.

H10 - NVIDIA Garak

Primary AI security harness Status: **PASS**

Purpose	LLM vulnerability scanner: probes (injection, jailbreak, encoding, leakage, toxicity...) plus detectors.
Repository	https://github.com/NVIDIA/garak branch `main` commit `cd87893be81bed4ced9afd86b88577289f4644bf`
License	Apache-2.0.
Release / activity	Latest release: v0.17.0 (2026-09-09). Last commit 2026-10-10. Stars: 9562. Active. v0.17.0 2026-09-09.
Installation	Python 3.12 venv `environments/garak`; `pip install -e .`
Installed version	0.17.1.pre1
Tests run	- L1 `--version` / `--help`: pass. - L2 `test.Test` probe vs `test.Blank`: PASS 40/40. - L3 local llama3.1:8b (<=20 prompts/probe): promptinject attack success 55%; encoding DecodeMatch 0%, DecodeApprox 10%.
AI models	Many generators incl. Ollama and local Hugging Face.
Capabilities	LLM red teaming; JSONL + HTML reports.
Risks	Some detectors download extra models; full sweeps are thousands of prompts.
Next step	Larger sweep against the local model (no API cost).

H11 - Microsoft PyRIT

Additional security / coding tool Status: **PASS**

Purpose	Python Risk Identification Tool: AI red-team orchestration with targets, converters, scorers and multi-turn attacks.
Repository	https://github.com/microsoft/PyRIT branch `main` commit `78bc1cb63695f6240751d0326a2ae0bad7cea50a`
License	MIT.
Release / activity	Latest release: v1.1.0 (2026-09-04). Last commit 2026-10-14. Stars: 4619. Very active. v1.1.0 2026-09-04 (source is 1.2.0.dev0).
Installation	Python 3.12 venv `environments/pyrit`; `pip install -e .`
Installed version	1.2.0.dev0
Tests run	- L1 import + `pyrit_scan --help`: pass. - L2 Base64Converter round-trip: pass (module is `pyrit.converter` in 1.x).
AI models	Azure/OpenAI and others; Ollama and LiteLLM references.
Capabilities	Multi-turn attack orchestration, converters, scorers, memory DB.
Risks	Do not attack AI systems you don't own.
Next step	Pin `pyrit==1.1.0` for stability; scripted attacks vs local model.

H12 - OpenHands

Additional security / coding tool Status: **PASS (L1)**

Purpose	General software-engineering agent platform; repo is now the Agent Canvas frontend/CLI and agent-server launcher.
Repository	https://github.com/OpenHands/OpenHands branch `main` commit `0e9ce191390f535347ff17055038c45eeef3fedf`
License	MIT.
Release / activity	Latest release: v1.27.1 (2026-10-12). Last commit 2026-10-13. Stars: 90839. Very active. v1.27.1 2026-10-12.
Installation	npm `--prefix environments/openhands` `@openhands/agent-canvas` (CLI only; never started).
Installed version	agent-canvas 1.27.1
Tests run	- L1 `agent-canvas --help`: pass (help path reviewed; does not start services).
AI models	Any via LiteLLM; ACP agents (Claude Code, Codex, Gemini...).
Capabilities	Coding agent, multiple sandboxes, automations, agent server API.
Risks	Option 1 (no sandbox) gives the agent full filesystem access. Option 2 mounts a projects dir into Docker.
Next step	Only with approval: Docker sandbox and a dedicated empty projects directory.

H13 - Aider

Additional security / coding tool Status: **PASS**

Purpose	AI pair-programming / code-editing CLI that commits changes to git.
Repository	https://github.com/Aider-AI/aider branch `main` commit `5dc9490bb35f9729ef2c95d00a19ccd30c26339c`
License	Apache-2.0.
Release / activity	Latest release: v0.86.0 (2025-08-09). Last commit 2026-05-22. Stars: 49531. Slowing: last commit 2026-05-22, last release v0.86 (2025-08).
Installation	`uv tool install --python 3.12 aider-chat` (lab-scoped).
Installed version	0.86.2
Tests run	- First install on Python 3.13 failed at runtime (pydub needs removed `audioop`); reinstalled on 3.12. - L1 `aider --version` / `--help`: pass.
AI models	Any via LiteLLM, incl. Ollama.
Capabilities	Patch generation with git integration; scripting mode.
Risks	Edits and commits code. Use only in throwaway copies.
Next step	Patch-only step in the approved-fix workflow.

H14 - Semgrep

Additional security / coding tool Status: **PASS**

Purpose	Rule-based static application security testing (SAST).
Repository	https://github.com/semgrep/semgrep branch `develop` commit `38a67a7eaf724fc266bf96e5705742bff8589775`
License	LGPL-2.1 engine; registry rules and Pro engine have separate terms.
Release / activity	Latest release: v1.180.0 (2026-10-07). Last commit 2026-10-07. Stars: 17039. Very active. v1.180.0 2026-10-07. Default branch `develop`.
Installation	`uv tool install semgrep`; metrics off, version check off, no cloud login.
Installed version	1.180.0
Tests run	- L1 `--version`: pass. - L2 9 local rules on synthetic fixture: 12 findings, 0 errors, all 9 seeded categories hit; JSON + SARIF written.
AI models	None (deterministic).
Capabilities	SAST, JSON/SARIF, CI.
Risks	Cloud features must stay disabled.
Next step	Grow the local rule pack; use as the deterministic baseline.

H15 - Opengrep

Additional security / coding tool Status: **PASS**

Purpose	Open-source fork of Semgrep's engine; Semgrep rule compatible.
Repository	https://github.com/opengrep/opengrep branch `main` commit `ffe40a1fa2f282e0a20d65c4dde8702bc45dad0c`
License	LGPL-2.1.
Release / activity	Latest release: v1.30.2 (2026-10-07). Last commit 2026-10-10. Stars: 3180. Active. v1.30.2 2026-10-07.
Installation	Official release binary `opengrep_osx_arm64` into `environments/opengrep`. install.sh reviewed but not used (writes to ~/.opengrep and ~/.local/bin). SHA-256 matched GitHub asset digest; cosign signature not verified.
Installed version	1.30.2
Tests run	- L1 `--version`: pass. - L2 same rules on fixture: 12 findings, identical to Semgrep.
AI models	None (deterministic).
Capabilities	SAST, JSON/SARIF, drop-in Semgrep rules (verified).
Risks	Install cosign to verify signatures.
Next step	Run alongside Semgrep for cross-checking.

H16 - OWASP ZAP

Additional security / coding tool Status: **BLOCKED**

Purpose	Dynamic application security testing (spider, passive and active scanning, automation framework).
Repository	https://github.com/zaproxy/zaproxy branch `main` commit `38ffe34a1b03fd3241e0bb233c2a083df875ee33`
License	Apache-2.0.
Release / activity	Latest release: v2.17.0 (2025-12-15). Last commit 2026-10-08. Stars: 15931. Active. v2.17.0 2025-12-15.
Installation	Not installed: Java 17+ missing and Docker daemon off.
Installed version	v2.17.0 (repo)
Tests run	- None possible without Java or Docker.
AI models	None.
Capabilities	DAST, REST API, HTML/XML/JSON reports (SARIF only via a separate add-on, not verified).
Risks	Active scanning only with approval against owned targets.
Next step	Approve Temurin JDK (Homebrew cask) or Docker image `zaproxy/zap-stable`.

H17 - LangGraph

Agent orchestration framework Status: **PASS**

Purpose	Stateful graph-based agent orchestration with checkpointing and human-in-the-loop interrupts.
Repository	https://github.com/langchain-ai/langgraph branch `main` commit `a6731f436700c7d46a9224ccd3a0e0ce3a5bfdb5`
License	MIT.
Release / activity	Latest release: cli==0.4.34 (2026-10-13). Last commit 2026-10-13. Stars: 43225. Very active.
Installation	Python 3.12 venv `environments/langgraph`; PyPI stable.
Installed version	1.2.15
Tests run	- L2 offline graph: scan -> human gate (`interrupt()`) -> report, with InMemorySaver checkpoint; resumed via `Command(resume=True)`; output asserted.
AI models	Any via LangChain integrations.
Capabilities	Checkpointing, approval gates, tool invocation, multi-agent.
Risks	Do not give agents arbitrary shell tools.
Next step	Recommended orchestrator once the plain controller needs approvals.

H18 - Microsoft AutoGen

Agent orchestration framework Status: **PASS**

Purpose	Multi-agent application framework (agentchat, core, ext).
Repository	https://github.com/microsoft/autogen branch `main` commit `027ecf0a379bcc1d09956d46d12d44a3ad9cee14`
License	Code MIT (LICENSE-CODE); docs CC-BY-4.0 (what GitHub detects).
Release / activity	Latest release: python-v0.7.5 (2025-09-30). Last commit 2026-04-06. Stars: 61393. Maintenance mode; README points to Microsoft Agent Framework.
Installation	Python 3.12 venv `environments/autogen`; autogen-agentchat + autogen-ext[openai] 0.7.5.
Installed version	0.7.5
Tests run	- L2 AssistantAgent with ReplayChatCompletionClient (no API): reply asserted.
AI models	Many, incl. Ollama extension.
Capabilities	Multi-agent teams, messaging, tools.
Risks	Maintenance mode: avoid for new builds.
Next step	Not recommended as the orchestration layer.

H19 - CrewAI

Agent orchestration framework Status: **PASS**

Purpose	Role-based multi-agent crews and flows.
Repository	https://github.com/crewAIInc/crewAI branch `main` commit `75808b0eca0ca17ef0cd578130d7055cd4c13f12`
License	MIT.
Release / activity	Latest release: 1.15.27 (2026-10-09). Last commit 2026-10-14. Stars: 59666. Very active. 1.15.27 2026-10-09.
Installation	Python 3.12 venv `environments/crewai`; PyPI; telemetry disabled.
Installed version	1.15.27
Tests run	- L1 `crewai version`: pass. - L2 Agent/Task/Crew construction with fake key; `kickoff()` not called (would call an LLM).
AI models	Many via LiteLLM, incl. Ollama.
Capabilities	Role-based agents, delegation, sequential/hierarchical processes.
Risks	Telemetry on by default; keep disabled.
Next step	Alternative to LangGraph; not needed for this pipeline.

6. Capability matrix

T = verified by a test in this lab; D = documented and present in source, not tested here; - = not a feature.

Tool	Vuln discovery	Static	Dynamic	LLM red team	Local models	Multi-agent	Outputs
VVAH	D	D	partial D	-	via OpenAI-compat	D	SARIF, JSON, MD
VulnHuntr	D (Python)	D	-	-	D (Ollama)	-	text
Strix	D	partial	D	-	D	D	reports
PentAGI	D	-	D	-	D	D	web UI
Shannon	D	partial	D (exploits)	-	partial	D	MD
PentestGPT	D	-	D	-	D (legacy)	partial	logs
SWE-agent	partial	-	-	-	D	-	trajectories
Inspect AI	-	-	-	D (scoring T)	D	D	.eval T
Promptfoo	-	partial	partial	D + L3 T	T (Ollama)	-	JSON T, HTML
Garak	-	-	-	D + L3 T	T (Ollama)	-	JSONL T, HTML T
PyRIT	-	-	-	D (converter T)	D	D	DB
OpenHands	partial	-	-	-	D	partial	conversations
Aider	-	-	-	-	D	-	git commits
Semgrep	T	T	-	-	n/a	-	JSON T, SARIF T
Opengrep	T	T	-	-	n/a	-	JSON T, SARIF T
ZAP	D	-	D	-	n/a	-	HTML/XML/JSON
LangGraph	-	-	-	-	D	D (gate T)	-
AutoGen	-	-	-	-	D	D (T replay)	-
CrewAI	-	-	-	-	D	D	-

7. Verified test results

Test	Result
Semgrep vs Opengrep on synthetic fixture (9 rules)	12 findings each, identical; SQLi x2, hardcoded secrets x3, pickle, unsafe YAML, mktemp, shell=True, eval, path traversal, admin route without auth
Inspect AI mock-LLM eval	status success, accuracy 1.0 (after tiktoken cache fix)
Garak test.Test vs test.Blank	PASS 40/40
Promptfoo echo provider	1/1 assertions, 0 tokens
PentestGPT unit tests	133 + 115 passed
LangGraph approval gate / AutoGen replay / PyRIT converter	all passed
Promptfoo vs llama3.1:8b (L3, local)	5/5 secret-leak attempts resisted
Garak vs llama3.1:8b (L3, local, 20 prompts/probe)	promptinject 55% attack success; encoding 0% (exact) / 10% (approx)

8. Top recommendations

Use case	Recommended
Source-code scanning	Semgrep / Opengrep baseline + VVAH (--stop-after s9)
AI red teaming	Promptfoo (app level) + Garak (model level), scored in Inspect AI; PyRIT for multi-turn
Remediation	Aider in patch-only mode on copies; SWE-agent once Docker is on
Orchestration	Plain Python controller with SARIF merge, then LangGraph for human approval gates
Level-4 only (isolated targets)	Strix, PentAGI, Shannon, PentestGPT, ZAP
Avoid for new work	AutoGen (maintenance mode), VulnHuntr (stale)

Suggested pipeline

Semgrep + Opengrep + VVAH -> merged SARIF -> triage -> human review gate (LangGraph interrupt) -> patch proposal (Aider / SWE-agent on a copy) -> regression tests -> report. AI-evaluation track runs separately: Promptfoo + Garak + PyRIT scored in Inspect AI. Pentest tools stay outside the pipeline behind a Level-4 authorisation boundary.

9. Costs and risks

- Static tools and local models: $0 API cost. VVAH estimated 252k-1M input tokens for S1-S9 on a 507-token fixture; multiply by the chosen model's price.
- Code-scanning agents send source code to the model provider; use the synthetic fixture or explicitly approved repos only.
- Shannon, PentAGI, Strix and PentestGPT execute real attacks; PentAGI's default config grants root Docker socket access.
- VVAH and PentestGPT auto-detect logged-in Claude/Codex CLIs, so they could spend quota without an explicit key. Keep a lab $HOME.
- Licensing: Shannon and VulnHuntr AGPL-3.0; PentAGI EULA; Semgrep/Opengrep LGPL-2.1.

10. Next steps (approval required)

1. Larger Garak/Promptfoo sweep against the local model ($0).
2. VVAH scan of the fixture with a chosen model and spend cap.
3. Start Docker Desktop to unblock Strix, SWE-agent, ZAP and PentAGI.

4. Pinned Shannon run (npx @keygraph/shannon@3.4.0 --help).
5. Java or ZAP Docker image for OWASP ZAP.
6. Build an isolated vulnerable target container before any Level-4 pentest.